Anomalies and the Expected Market Return

Forecasting with Machine Learning

Natasia Villadsen

2026-11-11

Introduction

Today

1	**The idea**	Anomalies, long-short portfolios, limits to arbitrage, and why anomaly returns could forecast the market
2	**The methods**	Seven forecasts of the market return, the expanding window, and how forecasts are evaluated
3	**The results**	Table II, Figures 1 and 2, and the gap between the short and the long leg, next to our replication

> Unlike in Lecture 5, the authors provide data and code, so a full replication is possible

Reading Guide

Introduction	pp. 639–644: the question, the findings, the economic mechanism and a preview of the methods.
Section I (optional)	pp. 644–651: a stylized model. Possibly read the first page, which gives the setup (Eqs. 1–3). The rest derives the sign of the predictive coefficient.
Section II	pp. 651–656: the seven models and how they are evaluated.
Sections III–IV	pp. 656–665: data (Table I) and the main results (Table II, Figures 1–3).
Section V (optional)	pp. 666–676: further evidence on limits to arbitrage (Tables III–V, Figures 4–5).
Section VI	pp. 676–677: conclusion.

Anomalies and the Expected Market Return (Dong et al., 2022)

Paper: “Anomalies and the Expected Market Return”, *Journal of Finance*, 77(1), 639–681, 2022

Authors:

Xi Dong Baruch College, City University of New York

Yan Li Southwestern University of Finance and Economics

David E. Rapach Saint Louis University

Guofu Zhou Washington University in St. Louis

Research question: **Do long-short anomaly returns predict the market excess return?**

The Authors' Main Findings

Shrinkage is necessary

OLS on all 100 anomaly returns overfits: its monthly market return forecasts go beyond ±6% in some months, and its R^2_{OOS} is −2,513.86%

Anomaly returns predict the market

The six regularized forecast models beat the prevailing mean (the average past market return). Their monthly R^2_{OOS} is 0.89% to 2.81%, above the 0.5% that Campbell and Thompson (2008) call economically meaningful, and the Clark and West test is significant for all of them

Economic value

A mean-variance investor would pay a fee of 2.59 to 6.38 percentage points of returns per year to use the forecasts (forgo returns of this amount to get the same utility as with the forecasts)

Slow correction of overpricing

The signal comes from the short leg (the overpriced stocks), and predictability is stronger when market frictions are high.
This aligns with overpricing being corrected slowly (explained on the next slides)

A short-horizon signal

Predictability falls with the horizon, with no significant evidence at 12 months: the correction of mispricing plays out over a few months

Characteristics and Anomalies

Characteristic

- A firm-level variable observed for every stock in every month
- Example: the **book-to-market ratio**, the book value of equity divided by its market value. High: cheap relative to the books (value). Low: expensive (growth)
- On its own it makes no claim about future returns

Anomaly

- A stock characteristic that has been documented to predict returns, which standard asset pricing models (e.g., the CAPM) do not explain
- Example: stocks that rose the most over the past year tend to keep outperforming stocks that fell the most (momentum)
- Buying past winners and selling past losers earns a significant alpha, the return not explained by market risk

> Every anomaly is a characteristic, but not every characteristic is an anomaly

The Long-Short Portfolio

- Each month, rank the stocks on a characteristic. The two extremes of the ranking are the **legs** of the portfolio
- **Long leg**: expect to earn high returns (cheap stocks): buy
- **Short leg**: expect to earn low returns (expensive stocks): sell (short). The shares are borrowed, sold now and bought back later, so the position gains if the price falls
- The short sale finances the purchases, so the net investment is approximately zero.

> Payoff = return of the long leg minus return of the short leg

Long-Short Payoffs Example

Month	Long leg	Short leg	Long-short payoff
Market booms	+8%	+6%	+2%
Market crashes	−5%	−7%	+2%
Ranking right, market flat	+3%	−3%	+6%
Ranking wrong	+1%	+4%	−3%

> Market moves hit both legs and cancel, so the payoff depends on the ranking.
> It is positive only on average (mean 0.38% per month, standard deviation 4.37%, Table I)

Value-Weighted Decile Sorting

1	**Rank**	At the beginning of the month, rank all stocks on the characteristic (here the book-to-market ratio), which is known before the month's returns
2	**Cut**	Split the ranking into 10 equal-sized groups (deciles). Decile 1 has the lowest values (expensive stocks, the short leg), decile 10 the highest (cheap stocks, the long leg)
3	**Weight**	Inside a decile, weight each stock by its market capitalization: value-weighted
4	**Hold**	Hold the decile portfolios for the month and record their returns. Next month, rank again

Decile	Stock	Book-to-market	Market cap (USD bn)	Return	Equal-weighted return	Value-weighted return
10	A	1.8	80	+4%		
10	B	2.2	20	+9%	6.5%	$0.8 \times 4 + 0.2 \times 9 = 5.0\%$
1	C	0.3	50	−2%		
1	D	0.4	50	+2%	0.0%	$0.5 \times (-2) + 0.5 \times 2 = 0.0\%$

> Long-short return: 5.0% − 0.0% = 5.0%.
> Value-weighting lets large firms dominate each portfolio, as in the market itself

Decile Sorting in Code I

Python Code | Start Over | Run Code

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(39)
stocks = pd.DataFrame({"book_to_market": rng.uniform(0.1, 2.5, 12).round(2),
                       "cap": rng.integers(5, 100, 12),              # market cap, USD bn
                       "ret": rng.normal(3, 3, 12).round(1)})        # next month's return, %

stocks["rank"] = stocks["book_to_market"].rank(method="first").astype(int) # 1. rank: 1 = most expensive
stocks["decile"] = np.ceil(stocks["rank"] / (len(stocks) / 10)).astype(int)   # 2. 10 equal groups
stocks.sort_values("rank")
```

	book_to_market	cap	ret	rank	decile
2	0.43	65	-1.5	1	1
6	0.46	96	1.5	2	2
7	0.65	50	0.0	3	3
3	0.84	26	1.6	4	4
0	1.30	42	1.2	5	5
1	1.39	25	4.4	6	5
4	1.49	31	3.1	7	6
11	1.68	97	2.7	8	7
9	1.70	58	3.4	9	8
10	1.94	37	2.0	10	9
5	2.00	26	4.0	11	10
8	2.36	20	2.6	12	10

Decile Sorting in Code II

Python Code | Start Over | Run Code

```python
# 3. weight = cap / total cap of the decile (e.g. 80 / 100 = 0.8)
stocks["weight"] = stocks["cap"] / stocks.groupby("decile")["cap"].transform("sum")
# 4. decile return = sum of weight x return (e.g. 0.8 x 4 + 0.2 x 9 = 5.0)
decile_ret = (stocks["weight"] * stocks["ret"]).groupby(stocks["decile"]).sum()

print(stocks[stocks["decile"].isin([1, 10])].sort_values("rank").round(2))
print("long leg (dec. 10):", round(decile_ret[10], 2), "  short leg (dec. 1):", round(decile_ret[1], 2))
print("long-short:", round(decile_ret[10] - decile_ret[1], 2))
```

```
   book_to_market  cap  ret  rank  decile  weight
2            0.43   65 -1.5     1       1    1.00
5            2.00   26  4.0    11      10    0.57
8            2.36   20  2.6    12      10    0.43
long leg (dec. 10): 3.39   short leg (dec. 1): -1.5
long-short: 4.89
```

- Decile 10 minus decile 1: long-short return for one anomaly in one month, one cell of the predictor table

Cross-Section versus Time Series

	Cross-sectional	Time series
Question	Which stocks beat which this month?	Will the market go up or down next month?
One observation	A stock, $r_{i,t+1}$	A month, $r_{M,t+1}$
Predictors	Characteristics $x_{i,t}$ of each stock (value, size, momentum)	Variables z_t of each month (valuation ratios, interest rates, inflation)
Finding	Anomalies: sorted portfolios earn average returns that standard models (CAPM, Fama-French) do not explain	Few predictors beat the prevailing mean out of sample (Goyal and Welch, 2008)
Example	Gu et al. (2020, Lecture 5): a panel, every stock in every month	This paper

> Anomalies are a cross-sectional phenomenon
>
> This paper asks whether they say anything about the market as a whole

How the Paper Links the Cross-Section to the Time Series

Cross-section
Month t: an anomaly and a return for every stock

→

Sort
Decile 10 minus decile 1: one long-short return $r^i_{LS,t}$

→

Time series
Every month, every anomaly: 100 series of long-short returns

→

Forecast
Regress $r_{M,t+1}$ on the 100 long-short returns of month t

> The paper takes the long-short returns of the cross-sectional literature and uses them as predictors in a time-series forecast of the market excess returns

Intuition: Limits to Arbitrage

- **Mispricing**: a price that deviates from fundamental value
- **Arbitrageurs** (e.g., hedge funds) buy underpriced and sell (short) overpriced stocks, which corrects the mispricing
- **Limits to arbitrage**: reasons why they cannot do this fully or immediately

COSTS

Trading costs and fees for borrowing shares

RISK

Prices can move further away before they converge, and losses can force early liquidation

LIMITED CAPITAL

Only a limited amount of money can be allocated to each mispriced stock

> The result: mispricing is corrected gradually, over several months.
> The paper calls this **mispricing correction persistence (MCP)**

Asymmetric Limits to Arbitrage

Underpriced stock: price 80$, worth 100$

- Buy the stock
- The most you can lose is 80$
- No borrowing, no fee

Overpriced stock: price 120$, worth 100$

- Must short it: borrow shares and pay a fee, if shares can be borrowed at all
- No limit on losses: the price can keep rising
- If the price rises to 140$ before it falls, losses keep growing and may force you to close the position
- Some investors are not allowed to short

Asymmetry: frictions are more significant when shorting than when going long
$\rightarrow$ Overpricing is corrected more slowly (Miller, 1977): stronger MCP for the short leg

From Frictions to Predictability

1	**Frictions**	Short-selling frictions make it hard to correct overpricing of the stocks in the short leg
2	**Slow correction**	Overpricing is corrected gradually: short-leg stocks that fell this month tend to fall again next month, because correction occurs slowly. The long leg is corrected quickly, so it carries little persistence
3	**Signal**	A falling short leg raises the long-short return this month. The long leg also moves it, but that part does not persist
4	**Market**	The short-leg stocks keep falling next month, and they are part of the market: the market return is lower

> In $r_{M,t+1} = \alpha + \beta\, r_{LS,t} + \varepsilon_{t+1}$, $\beta < 0$: a higher long-short return this month lowers the forecast of next month's market return

Why Should Anomaly Returns Predict the Market?

SLOW CORRECTION

Mispricing correction persistence

If overpricing is corrected slowly, a fall in the short leg today is followed by more falls next month, in the market too: $\beta < 0$ (Sec. I, Eq. 15)

LONG MINUS SHORT

Noise reduction

The two legs share a common component unrelated to mispricing (Sec. I, Eq. 1). Taking the difference removes it

MANY ANOMALIES

Aggregation

Each anomaly return also contains noise of its own (Sec. I, Eq. 19). The noise is nearly uncorrelated across anomalies (average correlation 0.08, Table I), so combining the 100 series, as all seven models do, cancels much of it

Methodology and Replication

Replication Plan

The authors' package provides data, R code, and output files

Step	Task	From the authors	Detail to get right
1 Data	Long-short anomaly returns and the market excess return	Finished returns (data files)	The raw data (CRSP, Compustat) is proprietary
2 Pre-processing	Impute NAs, lag predictors, build multi-period targets	R code	
3 Forecasting	Seven methods in an expanding window: one forecast per month, 1985:01–2017:12 (396 forecasts)	R code	Elastic net with AICc, OLS with dependent columns
4 Comparison	R^2_{OOS} and Clark-West statistics against Table II	Output files (forecasts and tables)	

Data and Forecasting Setup (Secs. II.A, III)

Target Next month's market excess return $r_{M,t+1}$: CRSP value-weighted return of all stocks minus the 1-month T-bill rate

Predictors The 100 long-short anomaly returns of month t, $r^i_{LS,t}$: decile 10 (long leg) minus decile 1 (short leg), value-weighted, rebalanced monthly

Anomalies From the cross-sectional literature, e.g., value, profitability, investment, issuance, momentum, and trading frictions

Stocks NYSE, AMEX and NASDAQ stocks, excluding stocks with a price below $5

Sample 1970:01–2017:12, 576 months per series

The predictable part of the monthly market return is small (low signal-to-noise ratio), so the prevailing mean is hard to beat (Goyal and Welch, 2008)

Table II also uses the 100 long-leg or the 100 short-leg anomaly returns as predictors instead

The Real Data in Code I

Python Code | Start Over | Run Code

```
import pandas as pd
import numpy as np

returns = pd.read_excel("assets/data/dataMarketRx.xlsx")
anomalies = pd.read_excel("assets/data/dataLongShortMissing.xlsx")

y = 100 * returns.iloc[:, 1].to_numpy()             # market excess return, %
X_na = 100 * anomalies.iloc[:, 1:].to_numpy()       # 100 anomaly returns, %, with NAs
X_all = np.where(np.isnan(X_na), np.nanmean(X_na, axis=1, keepdims=True), X_na)

# cont'd on next slide
```

```
<exec>:9: RuntimeWarning: Mean of empty slice
```

- Column 0 of both files holds the dates. `X_all` replaces each NA by the average of the other anomalies that month

The Real Data in Code II

Python Code | Start Over | Run Code

```
# cont'd from previous slide

# Converting to dataframe
data = pd.DataFrame(X_all, index=anomalies.iloc[:, 0], columns=anomalies.columns[1:])
data["target: market, month t+1"] = pd.Series(y, index=data.index).shift(-1)
data.iloc[:, [0, 1, 2, -2, -1]].round(2).head()
```

date	ABSACC	ACC	AGE	ZEROTOT	target: market, month t+1
19700130	5.69	2.78	6.80	11.43	5.13
19700227	0.91	-2.90	-0.14	-2.71	-1.06
19700331	6.95	4.04	2.63	5.72	-11.00
19700430	8.72	8.06	8.02	12.35	-6.92
19700529	3.17	9.58	1.12	4.59	-5.79

- Each column is a time series. The cross-sectional sorting is done already inside each cell

Benchmark versus Competing Forecasts (Sec. II.A)

- **Benchmark**, the prevailing mean: the average of the market returns observed up to t

$$\hat{r}^{\mathrm{PM}}_{M,t+1} = \frac{1}{t}\sum_{s=1}^{t} r_{M,s}$$

- **Competing forecasts**: regressions of next month's market return on this month's 100 anomaly returns

$$r_{M,t+1} = \alpha + \sum_{i=1}^{100} \beta_i \, r^{i}_{LS,t} + \varepsilon_{t+1}$$

- The seven competing forecasts use the anomaly returns in different ways: all 100 at once (OLS, ENet), one at a time (Combine, C-ENet), or one constructed predictor (Avg, PCR, PLS)

Training and Testing Samples (Sec. IV)

Step 1: 1970:01–1979:12	1980:01	First univariate forecast for C-ENet
Step 61: 1970:01–1984:12	1985:01	First evaluated forecast, all seven methods
Step 456: 1970:01–2017:11	2017:12	

- Expanding window, monthly re-estimation frequency. 396 forecasts, 1985:01–2017:12
- 1980–1984, holdout: needed only by C-ENet, which selects anomalies by the past accuracy of their univariate forecasts. Defaults to estimation sample for the remaining models
- No validation sample: AICc chooses the elastic net penalty

Dark: estimation sample. Light: the month that is forecast

Why Not OLS?

- Here: 101 coefficients, estimated from 179 months at the first evaluated forecast (1985:01)
- OLS fits the noise in the sample and forecasts it: $R^2_{\text{OOS}} = -2{,}513.86\%$ (Table II)

> The six other forecasts restrict/regularize the regression in different ways

The Seven Models (Sec. II.A)

Forecast	Construction	Shrinkage
OLS	Multiple regression on all 100 anomaly returns	None
ENet	The same regression with an elastic net penalty, size chosen by AICc	Penalty, sets some coefficients to zero
Combine	The average of 100 univariate regression forecasts, one per anomaly	Averaging
C-ENet	The average of the univariate forecasts that an elastic net selects	Selection, then averaging
Avg	Univariate regression on the average of the 100 anomaly returns	One equal-weighted predictor
PCR	Univariate regression on the first principal component	One component, ignores the target
PLS	Univariate regression on the first PLS component	One component, built to covary with the target

> All six alternatives to OLS shrink the forecast towards the prevailing mean

OLS in Code

Python Code

```python
def independent_columns(X):
    """Columns of X to keep when linearly dependent ones are dropped, in order (as lm in R)."""
    A = np.column_stack([np.ones(len(X)), X])
    tol = 1e-8 * np.abs(A).max() * max(A.shape)
    keep = [0]                                          # the intercept
    for j in range(1, A.shape[1]):
        if np.linalg.matrix_rank(A[:, keep + [j]], tol=tol) == len(keep) + 1:
            keep.append(j)
    return [j - 1 for j in keep[1:]]

def ols_forecaster(X_tr, y_tr, X_new):
    keep = independent_columns(X_tr)                    # drop linearly dependent columns, as lm() in R
    model = LinearRegression().fit(X_tr[:, keep], y_tr)   # estimate on the training months
    return model.predict(X_new[:, keep])[0]             # forecast for the next month
```

- `ols_forecaster()` and the functions to follow receive the training data `X_tr`, `y_tr` and the new predictors `X_new`, and return the one-step-ahead forecast

Elastic Net and AICc

$$\hat{\beta}_{\text{ENet}} = \arg\min_{\beta} \left\{ \sum_{t} \left(r_{M,t+1} - x_t^\top \beta \right)^2 + \lambda \left[\rho \sum_{j=1}^{100} |\beta_j| + (1-\rho) \sum_{j=1}^{100} \beta_j^2 \right] \right\}$$

- x_t: predictors (anomaly returns of month t), λ: penalty strength, ρ: mixing parameter. The ℓ_1 part shrinks the coefficients and sets some to zero, while the ℓ_2 part shrinks as well
- λ is chosen from a grid of 200 values by the *corrected* Akaike information criterion (AICc), with n observations and $\mathrm{df}(\lambda)$ non-zero coefficients,

$$\mathrm{AIC}(\lambda) = n \log\left(\mathrm{RSS}(\lambda)/n\right) + 2\,\mathrm{df}(\lambda)$$

$$\mathrm{AICc}(\lambda) = \mathrm{AIC}(\lambda) + \frac{2\,\mathrm{df}(\lambda)\left(\mathrm{df}(\lambda)+1\right)}{n - \mathrm{df}(\lambda) - 1}$$

> The AICc replaces hyperparameter tuning via validation sample. Select the λ minimizing the AICc

The authors use `glmnet` in R, with mixing parameter `alpha` = 0.5. It writes the penalty as $\lambda[\alpha\|\beta\|_1 + \frac{1}{2}(1-\alpha)\|\beta\|_2^2]$ and divides the squared errors by $2n$, so `alpha` = 0.5 is $\rho = 2/3$ above

Elastic Net in Code

Python Code

```
def enet_forecaster(X_tr, y_tr, X_new):
    intercept, coef = enet_aicc(X_tr, y_tr)
    return (intercept + X_new @ coef)[0] # Means intercept plus sum of coefficients times predictors
```

- The authors implement elastic net via `glmnet` in `R`
- We must implement this manually (`enet_aicc`), because `ElasticNet` from `sklearn` does not match the approach exactly (the forecasts differ from the paper in about 7% of the months)
- `enet_forecaster` function relies on `enet_aicc()` , omitted because it is very long – see lecture notebook

Combine: Average of Univariate Forecasts

$$\hat{r}_{M,t+1}^{\text{Combine}} = \frac{1}{100}\sum_{i=1}^{100}\left(\hat{\alpha}_i + \hat{\beta}_i\, r_{LS,t}^{i}\right)$$

- Each term is the forecast of a univariate regression of $r_{M,s+1}$ on $r_{LS,s}^{i}$, one per anomaly

Illustration with 4 anomalies

	Anomaly 1	Anomaly 2	Anomaly 3	Anomaly 4	Combine
Forecast of next month's market return	+0.9%	+0.3%	−0.2%	+0.5%	¼ (0.9 + 0.3 − 0.2 + 0.5) = +0.38%

> Each univariate forecast uses one noisy predictor. Averaging lets their errors offset, so *Combine* aids in isolating the signal from the noise

C-ENet: Choosing Which Forecasts to Average

1	**Collect**	In every earlier month, 100 univariate forecasts were made and the market returns were later realized. Store both. This history is why forecasting starts in 1980, before the evaluation
2	**Regress**	Regress the realized market returns on the 100 stored forecasts with an elastic net and non-negative coefficients (a Granger-Ramanathan regression)
3	**Select**	Keep anomalies whose forecasts receive a non-zero coefficient
4	**Average**	Average the forecasts – as made in the *Combine* approach – of the selected anomalies to produce the C-ENet forecast

The same 4 anomalies

	Anomaly 1	Anomaly 2	Anomaly 3	Anomaly 4	C-ENet
Coefficients from the regression	0.6	0	0.2	0	selects 1 and 3
Forecast	+0.9%	+0.3%	−0.2%	+0.5%	(0.9 + (−0.2)) / 2 = +0.35%

> Averaging all 100 variables regularizes strongly.
> C-ENet avoids over-shrinking by averaging only the selected anomalies

Combine and C-ENet in Code

Python Code

```
def univariate_forecasts(X_tr, y_tr, X_new):
    # One simple regression per anomaly: 100 forecasts
    forecasts = []
    for j in range(X_tr.shape[1]):              # loop over the 100 anomalies
        x_tr_j = X_tr[:, [j]]                   # anomaly j in the training months, as a column
        x_new_j = X_new[:, [j]]                 # anomaly j this month
        model = LinearRegression().fit(x_tr_j, y_tr)
        forecasts.append(model.predict(x_new_j)[0])
    return np.array(forecasts)

def cenet_forecaster(uni_tr, y_tr, uni_new, pm):
    # uni_tr: past univariate forecasts, y_tr: associated targets,
    # uni_new: current univariate forecasts, pm: prevailing mean
    _, coef = enet_aicc(uni_tr, y_tr, lower=0.0)
    selected = coef != 0
    return uni_new[0, selected].mean() if selected.any() else pm
```

- Combine is the mean of the 100 univariate forecasts, `uni_new.mean()`
- `lower=0.0` imposes non-negative coefficients
- If C-ENet selects nothing, the forecast is the prevailing mean `pm`

Principal Component Regression (PCR)

- With many predictors $x_{1,t}, \ldots, x_{p,t}$, we can compress them into $K \ll p$ constructed predictors, **the principal components**
- Each component is a weighted sum of the predictors: $Z_{k,t} = w_k^\top x_t$, $k = 1, \ldots, K$
- The weights are chosen such that the components retain as much of the information in the predictors as possible: Z_1 explains the largest share of their variation, Z_2 the largest share of what is left, and so on. The components are uncorrelated by construction
- The predictors for both PCR and PLS require standardization
- Regression of the target on the K components: $y_{t+1} = \alpha + \sum_{k=1}^{K} \beta_k Z_{k,t} + \varepsilon_{t+1}$
- In this paper, both PCR and PLS use **only the first component**, $K = 1$,

Tip

We do not go into the construction of the components, as it is rather involved. Refer to the book An Introduction to Statistical Learning in Python (ISLP), James et al., 2023, chapters 6.3, 6.4, and 12.1-12.4.

Partial Least Squares (PLS)

- Same idea as PCR: compress the predictors into K weighted sums $Z_{k,t} = w_k^\top x_t$, and regress the target on them
- Difference: the components retain as much of the information that is relevant *for the target* as possible. Z_1 moves the most with the target, Z_2 the most among what remains, etc. The components are uncorrelated by construction
- For Z_1, each predictor is weighted by how much it moves with the target. A predictor unrelated to the target gets a weight near zero
- Regression of the target on the K components: $y_{t+1} = \alpha + \sum_{k=1}^{K} \beta_k Z_{k,t} + \varepsilon_{t+1}$

> Unsupervised: PCR constructs components retaining information considering only the *predictors*
> Supervised: PLS retains the information in the predictors relevant for the *target*

Dimension Reduction

The 100 anomaly returns are collapsed into one predictor Z_t, followed by a univariate regression

$$r_{M,t+1} = \alpha + \beta Z_t + \varepsilon_{t+1}, \qquad Z_t = \sum_{i=1}^{100} w_i \, r^i_{LS,t}$$

MODEL *AVG*

Equal weights

$w_i = 1/100$: Z_t is the (cross-sectional, i.e., row-wise) average of the anomaly returns

MODEL *PCR*

Principal components: largest variance

Weights chosen so that Z_t has the largest variance explained, reducing dimensionality of the data while retaining as much information as possible. Does not account for the target

MODEL *PLS*

Partial least squares: most relevant for the target

Weights chosen so that Z_t covaries as much as possible with the market return, retaining the information in the anomalies that is relevant for the target. Accounts for the target

Dimension Reduction in Code

Python Code

```
t = 180                                                    # months observed
S = StandardScaler().fit_transform(X_all[:t])              # standardized anomalies
A = np.nanmean(X_na[:t], axis=1, keepdims=True)            # average of the 100 anomalies in each month

def avg_forecaster(A_tr, y_tr, A_new):
    model = LinearRegression().fit(A_tr, y_tr)        # regress on the average
    return model.predict(A_new)[0]

def pcr_forecaster(S_tr, y_tr, S_new):
    pca = PCA(n_components=1).fit(S_tr)               # fitting the model
    Z_tr, Z_new = pca.transform(S_tr), pca.transform(S_new) # obtaining the first principal component
    model = LinearRegression().fit(Z_tr, y_tr)        # regress on the component
    return model.predict(Z_new)[0]

def pls_forecaster(S_tr, y_tr, S_new):
    pls = PLSRegression(n_components=1, scale=False).fit(S_tr, y_tr)   # component and regression in one
    return pls.predict(S_new).item()
```

- `S` : standardized anomaly returns, so that no anomaly dominates PCR and PLS due to its scale
- `A` : the one predictor of Avg, the average of the 100 anomaly returns in each month

Implementing All Models for One Month I

Python Code

```
def forecast_all(t, h, uni):
    i = t - R                                                  # row of uni for month t
    y_all = target(t, h)                                       # the targets known at month t
    y_tr = y_all[1:]
    pm = y_all.mean()                                          # pm: the prevailing mean (benchmark)
    # Window function: extracts training windows accounting for multi-period targets/longer horizon forecasting
    X_tr, X_new = window(X_all[:t], t, h)                      # the 100 anomaly returns
    S_tr, S_new = window(StandardScaler().fit_transform(X_all[:t]), t, h)    # standardised
    A_tr, A_new = window(np.nanmean(X_na[:t], axis=1, keepdims=True), t, h)  # their average
    uni_tr, uni_new = uni[:i - (h - 1)], uni[i:i + 1]          # univariate forecasts: past and current
    # cont'd on next slide
```

- `t` : the number of months observed when the forecast is made
- `h` is the horizon of the forecast in months
- `R` : the 120 months of the initial estimation sample
- `uni` has index initializing at `t = R` , meaning that month `t` has row `i = t - R`
- `uni` : the 100 univariate forecasts of every month after the initial estimation window, one row per month

Implementing All Models for One Month II

Python Code

```
# cont'd from previous slide
    return {"OLS": ols_forecaster(X_tr, y_tr, X_new),
            "ENet": enet_forecaster(X_tr, y_tr, X_new),
            "Comb": uni_new.mean(),
            "CENet": cenet_forecaster(uni_tr, y_all[R:], uni_new, pm),
            "Avg": avg_forecaster(A_tr, y_tr, A_new),
            "PCR": pcr_forecaster(S_tr, y_tr, S_new),
            "PLS": pls_forecaster(S_tr, y_tr, S_new),
            "PM": pm}                                    # the benchmark, kept next to the forecasts
```

- Only data up to time `t` enters, so every forecast is out of sample

Implementing All Models for the Full Sample

Python Code

```
R, hold, h = 120, 60, 1                 # initial window, holdout (months), horizon
names = ["OLS", "ENet", "Comb", "CENet", "Avg", "PCR", "PLS"]   # the seven methods
t_range = range(R + hold, len(y))       # t = 180, ..., 575: forecasts of 1985:01, ..., 2017:12

uni = past_univariate_forecasts(h)      # univariate forecasts for C-ENet (notebook)
results = Parallel(n_jobs=-1)(delayed(forecast_all)(t, h, uni) for t in t_range)
df = pd.DataFrame(results)              # one row per forecast month, one column per method

forecasts = df[names]                   # the seven forecasts
benchmark = df["PM"].to_numpy()         # the prevailing-mean forecast
realized = y[R + hold:]                 # the actual market return of each forecast month
```

- A forecast only uses data up to its month `t`, so the 396 forecasts are independent
- `past_univariate_forecasts(h)`: runs `univariate_forecasts` in every month after the initial window and stacks the results in `uni`, one row per month
- The 396 forecasts take a few minutes to compute, so we run them in parallel (`Parallel` (joblib) computes them simultaneously on different processor cores)
- `realized`: the actual market return of each forecast month

Up to you

1	**Run**	Start the notebook with `horizons = [1]` while we continue
2	**Compare**	Check each row of Table II against your output once the run is done
3	**Consider**	Which forecasts do you expect to be significant, and why does OLS fail?

Evaluation: R^2_{OOS} (Sec. II.B)

$$R^2_{\text{OOS}} = 1 - \frac{\sum_t \left(r_{M,t} - \hat{r}_{M,t}\right)^2}{\sum_t \left(r_{M,t} - \bar{r}_{M,t}\right)^2}$$

- The sums run over the 396 forecast months, 1985:01–2017:12. $\hat{r}_{M,t}$ is the competing forecast and $\bar{r}_{M,t}$ the prevailing mean, both made with data up to month $t-1$

> The benchmark fits the application: the prevailing mean for the market return, zero for individual stock returns (Gu et al., 2020)

Evaluation: The Clark-West Test (Sec. II.B)

- The competing models nest the benchmark: with all coefficients at zero, it is the prevailing mean
- Even if the true coefficients are zero, they are estimated with noise, which inflates the forecast errors
- The Diebold-Mariano test compares the two MSEs directly. Under equal predictive ability, the noise makes the benchmark look better
- The test is **undersized**: it rejects less often than it should and misses real predictability
- Clark and West offset the noise penalty, and regress this *adjusted* loss differential on a constant

$$f_t = \left(r_{M,t} - \bar{r}_{M,t}\right)^2 - \left(r_{M,t} - \hat{r}_{M,t}\right)^2 + \left(\bar{r}_{M,t} - \hat{r}_{M,t}\right)^2 = \mu + \varepsilon_t$$

- H_0: $\mu \leq 0$, no better than the prevailing mean. H_1: $\mu > 0$, better than the prevailing mean. One-sided t-test with HAC (Newey-West) standard errors
- Caveat: critical values are only approximate, justified by simulations. The test is modestly undersized (Clark and West, 2007)
- Implementation code is in the notebook

Evaluation: Economic Value (Sec. II.C)

- Does a better forecast translate to monetary gains? Consider an investor who each month splits wealth between the market and a risk-free asset
- **Mean-variance investor**: values a portfolio by two numbers only, its average return and its variance (returns: gains/losses, variance: risk)
- γ expresses risk aversion, how strongly the investor dislikes risk

The share of wealth in the market is,

$$w^*_{t+1|t} = \frac{1}{\gamma} \times \frac{\text{forecast return}}{\text{forecast variance}} = \frac{1}{\gamma} \times \frac{\hat{r}_{M,t+1|t}}{\hat{\sigma}^2_{t+1|t}}$$

Forecast $\hat{r}$ (volatility 4.5%)	Weight w^*	Position
0.6%	0.99	almost all wealth in the market
0.3%	0.49	half in the market
-0.5%	-0.82	short the market

> Only $\hat{r}$ differs between the model and the prevailing mean, so better forecasts give better weights

Scoring the Investors: Sharpe Ratio and Utility Gain

The investor's excess return in month $t+1$: $r_{p,t+1} = w^*_{t+1|t}\, r_{M,t+1}$, weight times market excess return

Sharpe ratio

- Average return / standard deviation of return, per year
- Return per unit of risk
- Example: 8% return, 16% volatility: Sharpe 0.5
- Paper: prevailing mean 0.45, ENet 0.81

Utility gain

- Utility = average return $-$ penalty for risk $\left(\frac{\gamma}{2} \times \text{variance}\right)$: a risk-adjusted return
- Gain: the increase in yearly utility when the investor uses the model instead of the prevailing mean, i.e., the annual fee the investor would pay to use the model instead of the prevailing mean
- Example: ENet gain 6.26%. The investor would give up 6.26% of return per year to use ENet's forecasts instead of the prevailing mean

$$\text{Sharpe} = \sqrt{12}\,\frac{\bar{r}_p}{\sigma_p} \qquad U = \bar{r}_p - \frac{\gamma}{2}\,\sigma_p^2 \qquad \text{Gain} = 12\left(U_{\text{model}} - U_{\text{PM}}\right)$$

$\bar{r}_p$, σ_p: monthly average and standard deviation of $r_{p,t+1}$. $\sqrt{12}$ and 12 annualize the Sharpe ratio and the utility

> Higher Sharpe ratio and positive utility gain: the forecast is worth money, not just a better R^2_{OOS}

Economic Value in Code

Python Code

```
gamma, var_window, w_min, w_max = 3, 60, -1, 2     # risk aversion, variance window, weight constraints

t_ev = np.arange(R + hold, len(y))                 # 396 forecast months

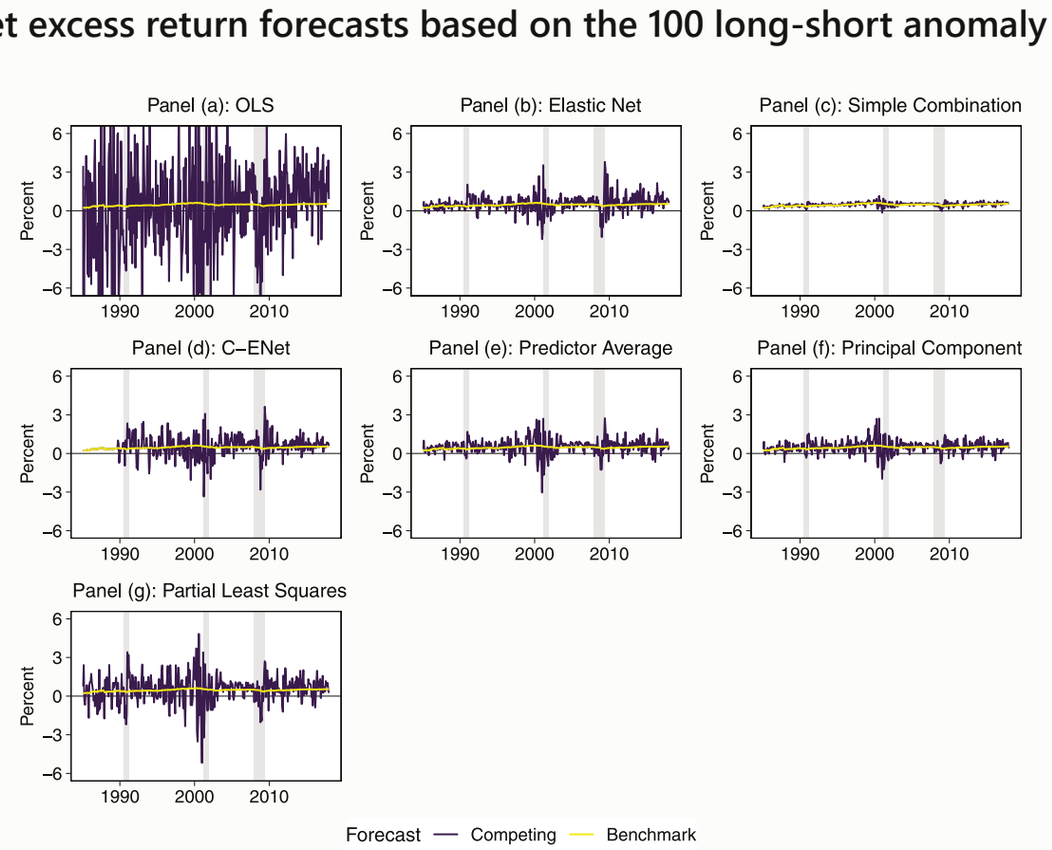
def weights(forecast):
    # Equity weight and excess return of the mean-variance investor (h = 1)
    variance = []
    for t in t_ev:
        past = y[t - var_window:t]                 # the 60 months before the forecast
        variance.append(past.var(ddof=1))          # their sample variance
    variance = np.array(variance)

    weight = np.clip(100 * forecast / (gamma * variance), w_min, w_max)      # w = forecast / (gamma * variance)
    return weight, weight * y[t_ev]                                          # weights and r_p, excess returns

def utility_and_sharpe(r_p):
    r = r_p / 100
    utility = r.mean() - 0.5 * gamma * r.var(ddof=1)  # mean minus risk penalty
    sharpe = np.sqrt(12) * r.mean() / r.std(ddof=1)   # annualized Sharpe ratio
    return utility, sharpe
```

Results in the Paper

Figure 1: The Forecasts



Source: Dong et al. (2022), Figure 1. Purple: the model forecast. Yellow: the prevailing mean. Grey bars: NBER recessions. 1985:01–2017:12

- OLS oscillates beyond ±6% (overfitting)
- The other six models are shrunk towards the prevailing mean (benchmark), and are more volatile around recessions

Table II: R^2_{OOS} of the Market Excess Return Forecasts

Predictors	OLS	ENet	Combine	C-ENet	Avg	PCR	PLS
Long-short return	−2,513.86	2.03**	0.89***	2.81***	1.89**	1.25**	2.06***
Long-leg excess return	−344,960.22	−0.90	0.29	−0.68	0.26	0.24	0.41
Short-leg excess return	−13,284.68	1.81*	0.72*	0.39*	0.75*	0.74*	0.84*

Monthly R^2_{OOS} in percent, 1985:01–2017:12. *, **, ***: Clark and West test at the 10%, 5% and 1% level

- OLS: 101 coefficients from 179 months at the first evaluated forecast, so it fits noise and its forecasts oscillate wildly (overfitting, Figure 1)
- Long-short returns forecast better than either leg alone
- The short leg has some predictive power (all six forecasts significant at the 10% level), the long leg exhibits no significant predictive power

Our Replication vs. Table II

Monthly R^2_{OOS} (percent) of the long-short anomaly returns, 1985:01–2017:12

	OLS	ENet	Combine	C-ENet	Avg	PCR	PLS
Ours	−2,513.86	2.03**	0.89***	2.81***	1.89**	1.25**	2.06***
Paper	−2,513.86	2.03**	0.89***	2.81***	1.89**	1.25**	2.06***

*, **, ***: Clark and West test at the 10%, 5% and 1% level. Ours: from the t-statistics with the one-sided critical values 1.28, 1.64 and 2.33

> All seven R^2_{OOS} statistics and all significance levels agree with Table II

Other Horizons: Replication and Paper

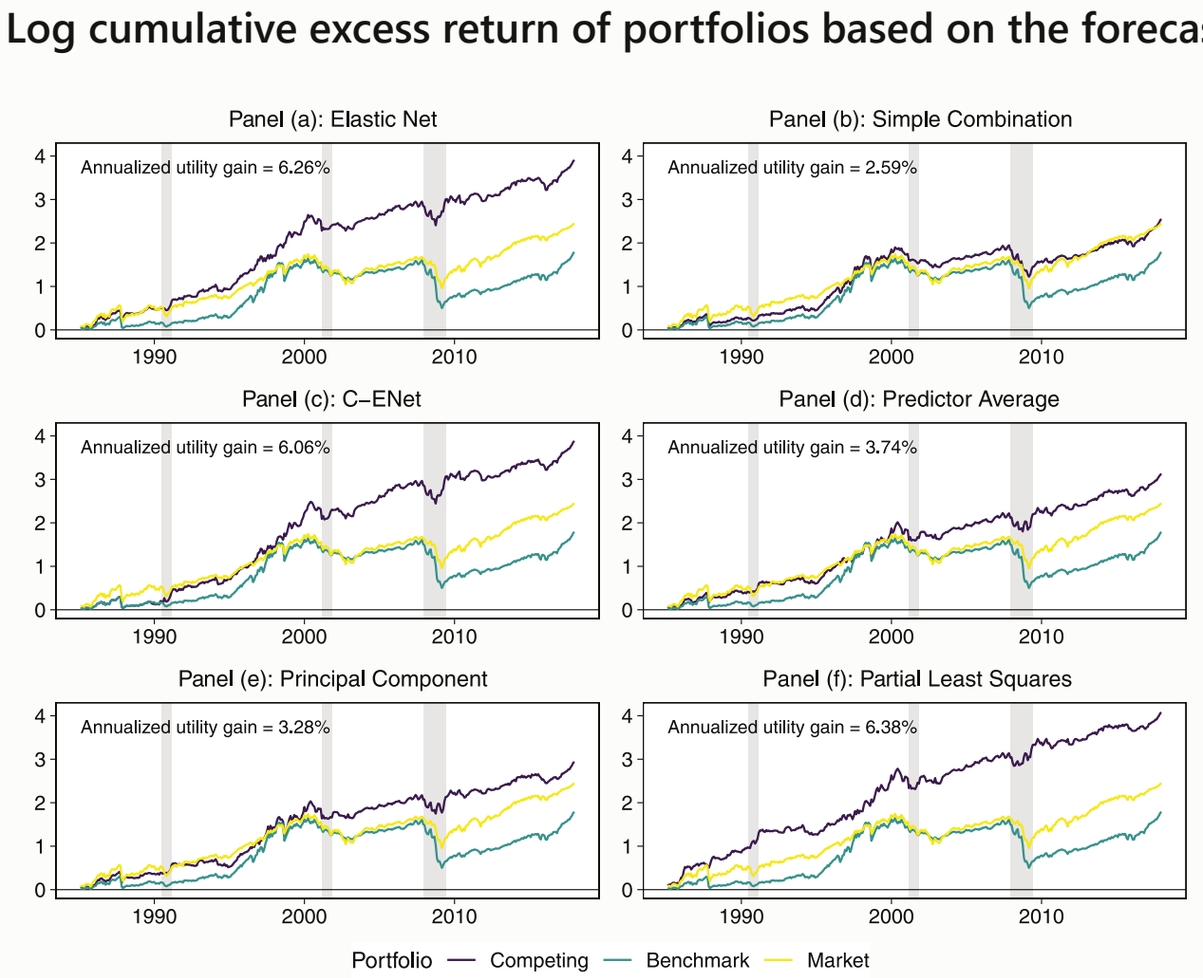
Monthly R^2_{OOS} (percent) by forecast horizon h in months: **ours (paper)**

Method	$h = 1$	$h = 2$	$h = 3$	$h = 6$	$h = 12$
ENet	2.03 (2.03**)	2.47 (2.47***)	0.55 (0.55***)	−0.47 (−0.47)	−2.11 (−2.11)
Combine	0.89 (0.89***)	0.54 (0.54**)	0.57 (0.57***)	0.61 (0.61***)	0.09 (0.09)
C-ENet	2.81 (2.81***)	0.08 (0.08**)	0.85 (0.85**)	0.77 (0.77)	−0.51 (−0.51)
Avg	1.89 (1.89**)	1.01 (1.01**)	1.44 (1.44**)	1.36 (1.36**)	−0.35 (−0.35)
PCR	1.25 (1.25**)	0.60 (0.60*)	0.36 (0.36)	0.66 (0.66*)	−0.78 (−0.78)
PLS	2.06 (2.06***)	0.17 (0.17**)	−0.16 (−0.16)	1.32 (1.32***)	−5.94 (−5.94)

Paper: Table II for $h = 1$, Internet Appendix Table IA.X for the other horizons. Significance from the Clark and West test (*, **, ***: 10%, 5%, 1%). OLS is far below zero at every horizon (−2,513.86 at $h = 1$ to −188,308.44 at $h = 6$) in both

> Predictability fades with the horizon, and no R^2_{OOS} is significant at $h = 12$

Figure 2: Cumulative Portfolio Returns



Source: Dong et al. (2022), Figure 2, 1985:01–2017:12. Purple: investor using model forecast. Teal: investor using prevailing-mean forecast. Yellow: investor holding the market.

Economic Value: Replication and Paper

Annualized Sharpe ratio and utility gain (percent per year)

	Prevailing mean	OLS	ENet	Combine	C-ENet	Avg	PCR	PLS	Market
Sharpe, ours	0.42	0.23	0.81	0.59	0.79	0.65	0.62	0.80	0.57
Sharpe, paper	0.45	0.23	0.81	0.59	0.79	0.65	0.62	0.80	–
Gain, ours		−4.97	6.26	2.59	6.06	3.74	3.28	6.38	
Gain, paper		−4.97	6.26	2.59	6.06	3.74	3.28	6.38	

Paper: Figure 2 and Internet Appendix Table IA.IV (our numbers are compared with the paper's reported values, not with the authors' output files). The paper reports no Sharpe ratio for the market

- Every shrinkage forecast beats the prevailing-mean investor, and the OLS does worst overall
- All gains and Sharpe ratios match, except the Sharpe ratio of the prevailing-mean investor (0.42 against 0.45), which we cannot explain. The bootstrap tests are not replicated

Wrapping Up

Conclusion of Replication

FORECASTS

Table II matches exactly

All seven forecasts match Table II, and the other horizons match the authors' output files

ECONOMIC VALUE

Figure 2 matches, one exception

Utility gains and Sharpe ratios match Figure 2 and Table IA.IV, except the Sharpe ratio of the prevailing-mean investor (0.42 against 0.45)

DETAILS MATTER

Exact match needs R's solver

Exact agreement needed the coordinate descent of `glmnet` for the elastic net and an OLS that drops dependent columns, as `lm` does (see the appendix)

The authors' output files tell us when we get it right, which was not possible for Gu et al. (2020)

Not replicated: the bootstrap tests of the utility gains, the long-leg and short-leg rows of Table II, the limits-to-arbitrage evidence (Sec. V), and the construction of the anomaly returns from CRSP, Compustat and I/B/E/S (we start from the finished anomaly long-short returns)

> Ultimately, **the replication is a success**: we match the paper's results

References

- Campbell, J. Y. and Thompson, S. B. (2008). Predicting excess stock returns out of sample. *Review of Financial Studies*, 21(4), 1509–1531. doi:10.1093/rfs/hhm055
- Clark, T. E. and West, K. D. (2007). Approximately normal tests for equal predictive accuracy in nested models. *Journal of Econometrics*, 138(1), 291–311. doi:10.1016/j.jeconom.2006.05.023
- Dong, X., Li, Y., Rapach, D. E. and Zhou, G. (2022). Anomalies and the expected market return. *Journal of Finance*, 77(1), 639–681. doi:10.1111/jofi.13099
- Replication package of Dong et al. (2022): data, R code and output files, in the supporting information of the article at the link above (folder "Replication code-20200266")
- Goyal, A. and Welch, I. (2008). A comprehensive look at the empirical performance of equity premium prediction. *Review of Financial Studies*, 21(4), 1455–1508. doi:10.1093/rfs/hhm014
- Gu, S., Kelly, B. and Xiu, D. (2020). Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5), 2223–2273. doi:10.1093/rfs/hhaa009
- James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J. (2023). *An Introduction to Statistical Learning: with Applications in Python*. Springer Texts in Statistics. Springer, Cham. statlearning.com
- McLean, R. D. and Pontiff, J. (2016). Does academic research destroy stock return predictability? *Journal of Finance*, 71(1), 5–32. doi:10.1111/jofi.12365
- Shleifer, A. and Vishny, R. W. (1997). The limits of arbitrage. *Journal of Finance*, 52(1), 35–55. doi:10.1111/j.1540-6261.1997.tb03807.x
- Miller, E. M. (1977). Risk, uncertainty, and divergence of opinion. *Journal of Finance*, 32(4), 1151–1168. doi:10.1111/j.1540-6261.1977.tb03317.x
- Friedman, J., Hastie, T. and Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22. doi:10.18637/jss.v033.i01

Appendix

Multi-Period Targets

- For horizon $h > 1$ the target is the average market return over the next h months. The paper uses $h = 1, 2, 3, 6, 12$, the main results are $h = 1$
- The training sample loses $h - 1$ additional rows, because their targets are not yet complete

Example at $t = 6$ and $h = 2$:

Training rows	Training targets	Forecast from	Target of the forecast
months 1–4	$\frac{1}{2}(r_2 + r_3), \ldots, \frac{1}{2}(r_5 + r_6)$	month 6	$\frac{1}{2}(r_7 + r_8)$

> Needed only for the horizon results (see *Other Horizons*): the signal fades after a few months

Elastic Net as in glmnet

- The authors' code uses R's `glmnet` with `alpha` = 0.5 and 200 values of λ
- The λ grid starts at the smallest value that sets all coefficients to zero and decreases geometrically, down to 10^{-4} times the start (or 10^{-2} if there are more predictors than observations)
- At each λ, coordinate descent cycles over the active variables until the largest squared change in a coefficient is below 10^{-7}
- The path stops when the fit stops improving (glmnet's rule). We also stop when AICc has not improved for 100 values of λ, which does not change the result

`enet_aicc(X, y)` is defined in the notebook, in the section "Elastic net as in R's `glmnet`". It returns the intercept and the coefficients at the AICc-optimal λ, and `lower=0` imposes non-negative coefficients. ENet and C-ENet call it

Why Not scikit-learn's ElasticNet?

- AICc picks the penalty from the number of non-zero coefficients, df
- The `ElasticNet` of scikit-learn solves the same problem, but it stops at a different tolerance and cycles through the variables in a different order
- That changes which coefficients are exactly zero, hence df, hence the selected λ. The forecasts then differ from the paper's in about 7% of the months
- The fix is to re-implement the coordinate descent of R's `glmnet` (same λ grid, threshold, active-set cycling and stopping rule), as on the previous slide

> Same lesson as in Lecture 5: the solver can decide the result

The Model Behind the Idea (Sec. I)

Each leg's return = a common part + a change in mispricing:

$$r_{l,t} = f_t + \Delta u_{l,t} \;\; (l = L, S), \qquad r_{LS,t} = \Delta u_{L,t} - \Delta u_{S,t}$$

- f_t: market-wide news, common to both legs. It cancels in the long-short return
- $u_{L,t} \leq 0$: underpricing (long leg). $u_{S,t} \geq 0$: overpricing (short leg)
- In $r_{M,t+1} = \alpha + \beta\, r_{LS,t} + \varepsilon_{t+1}$, $\beta < 0$ when

$$\operatorname{cov}(\Delta u_{S,t+1}, \Delta u_{S,t}) > \operatorname{cov}(\Delta u_{L,t+1}, \Delta u_{L,t})$$

- That is, the correction of overpricing persists more from month to month than the correction of underpricing

Paper pp. 644–651, Eqs. (1), (2), (13), (15). Noise reduction (Eqs. 16–17) and aggregation (Eqs. 19–22): see "Why Should Anomaly Returns Predict the Market?"